

An Open-Source Toolbox for Embedding Neural Stability Surrogates in Stability-Constrained Optimisation

An end-to-end coding tutorial

Prof. Fei Teng | Dr. Wangkun Xu

Department of Electrical and Electronic Engineering
Imperial College London

The 3rd International Summer School on IBR-dominated Power Systems
4 September 2026

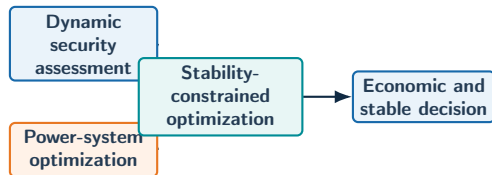
Dynamic security assessment and stability-constrained optimization

Physics-based DSA

- evaluates nonlinear DAEs and a stability index at z ;
- is accurate, but **slow**.

Data-driven DSA

- learns $\psi(z; \theta)$ from simulated operating points to the stability index;
- needs offline training;
- Main focus: Neural Network surrogate, which captures nonlinear relationships and evaluates **fast**.



SCO adds learned stability surrogate as constraint to power-system optimization.

A learned stability model becomes an operating constraint

$$\min_x f(x, d)$$

$$\begin{aligned} \text{s.t. } & g(x, d) \leq 0, \\ & z = T(x, d), \\ & \psi(z; \theta) \leq \phi_c. \end{aligned}$$

x optimization decisions

d fixed load and available solar

g conventional operating constraints

z neural-network features linked to x, d

ψ learned stability index

ϕ_c required stability threshold

This coding tutorial will cover end-to-end workflow for learning and embedding a neural-network-based small-signal stability constraint in a unit commitment.

Source: Adapted from Xu, Chu and Teng [1].

Open the tutorial notebooks before we begin

Repository

[Neural-SmallSignal-UC-Tutorial](#)

Recommended: Google Colab

- ① Open the repository README.
- ② Click the first notebook's Colab badge.
- ③ Run the **GOOGLE COLAB ONLY** setup cell once.

Two executable notebooks

`small_signal_concept_with_andes.ipynb`

Inspect the SG-GFL model, solve power flow and calculate eigenvalues.

`stability_constrained_optimization_with_ncet.ipynb`

Generate data, train the regressor, solve UC and verify the solutions in ANDES.

Keep both notebooks open and run them alongside the slides.

Source: Authors' tutorial repository and README.

Four packages connect simulation, learning and optimization

ANDES	Run AC power flow, dynamic initialization and small-signal eigenvalue analysis.
PyTorch	Train the ReLU regressor that predicts the critical eigenvalue real part.
CVXPY	Solve the baseline and stability-constrained unit commitment problems.
NCET	Encode the trained PyTorch graph as exact CVXPY MILP constraints.



All packages are open-source and free to use.

Sources: ANDES [2], NCET [3], CVXPY [6] and PyTorch [7].

The tutorial uses switchable SGs and one GFL resource

ANDES case: ieee14/ieee14_regcp1.xlsx

System base: 100 MVA

Location	Device	Tutorial treatment
Bus 1	GENROU Slack SG	Always online; balances power and AC losses
Buses 2, 3, 6	GENROU SGs	Commitment and active dispatch vary
Bus 8	REPCA1 + REECA1 + REGCP1 + PLL1	GFL remains online; active injection varies
Load buses	Constant-PQ loads	Fixed bus shares and power factors; total load varies

Tutorial weak-grid setting

Line 20 reactance:

$$0.17615 \rightarrow 0.792675 \text{ pu}$$

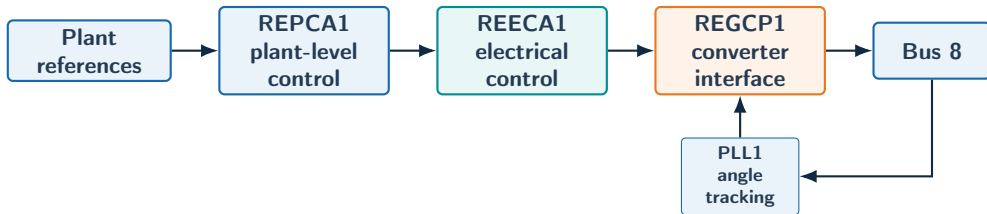
GFL maximum active power:

$$P_s^{\max} = 100 \text{ MW}$$

Controller parameters remain fixed.

Source: Authors' configuration using ANDES [2].

The GFL model separates control, conversion and synchronization

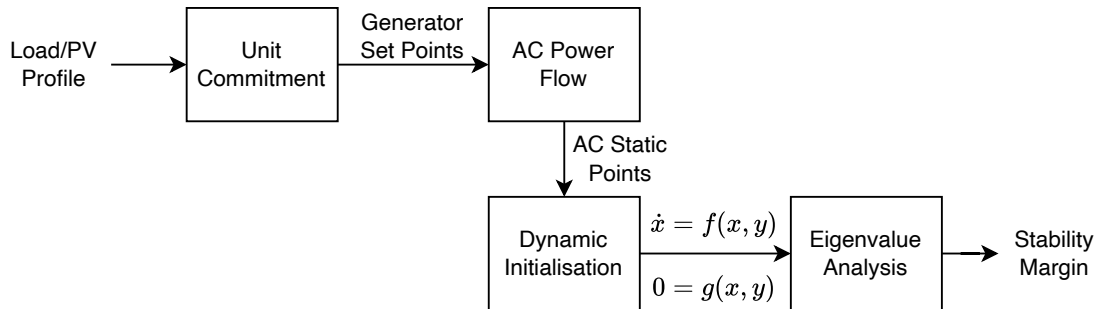


- **REPCA1**: plant level voltage/reactive control.
- **REECA1**: electrical control with current commands and current-limit logic.
- **REGCP1**: inverter control for current injection.
- **PLL1**: synchronization with measured bus angle.

Together, the four models represent one aggregated GFL resource at bus 8.

Sources: Fang et al. [4]; EPRI [5].

One operating point follows three ANDES stages



Source: Authors' diagram and implementation using ANDES [2].

ANDES performs AC power flow, dynamic initialization, and eigenvalue analysis

```
system = ss.build_system(point)
slack_mw = ss.solve_power_flow(system)
result = ss.calculate_small_signal(system, slack_mw)
```


Slack, PV and PQ buses specify different power-flow quantities

Bus type	Specified	Calculated	Typical device
Slack	V, θ	P, Q	Bus-1 reference generator
PV	P, V	θ, Q	SGs at 2, 3, 6; GFL at 8
PQ	P, Q	V, θ	Constant-power loads

$$\sum_{k \in \mathcal{D}_i} P_k = P_i^{\text{network}}(V, \theta), \quad \sum_{k \in \mathcal{D}_i} Q_k = Q_i^{\text{network}}(V, \theta)$$

- Inputs: Slack.v0/a0, PV.p0/v0, PQ.p0/q0.
- Bus.v/a store the shared voltage solution.
- A PV generator and PQ load may share one Bus.
- ANDES sums every connected device in the nodal balance.

Source: ANDES power-flow conventions [2].

The rightmost eigenvalue defines the stability target

Small-signal analysis:

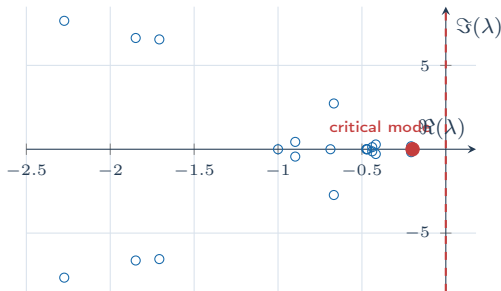
$$\dot{x} = f(x, y), \quad 0 = g(x, y)$$

$$A = f_x - f_y g_y^{-1} g_x, \quad \alpha = \max_{|\lambda_i| > \text{tol}} \Re(\lambda_i)$$

$$\text{label} = \begin{cases} 0, & \alpha \leq -10^{-4} \quad \text{stable} \\ 1, & \alpha > -10^{-4} \quad \text{unstable/margin} \end{cases}$$

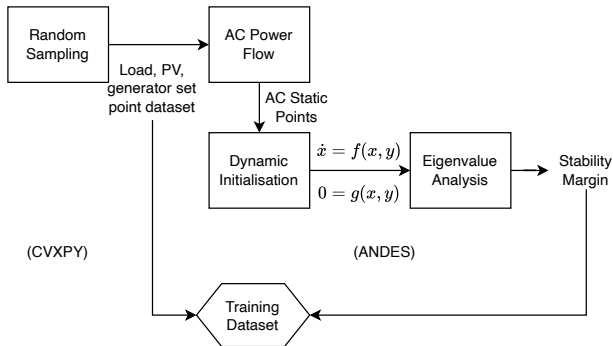
Example: $(u_2, u_3, u_6) = (1, 1, 1)$, $P_g = (10, 10, 30)$

MW and $P_s = 90$ MW yield $\alpha = -0.198388 \text{ s}^{-1}$.



Source: Authors' calculation using ANDES [2]; reproduced by the first notebook.

Dataset generation repeats the ANDES calculation



- 1 Enumerate seven nonzero SG commitments.
- 2 Sample SG dispatch, solar and total load.
- 3 Pre-screen approximate lossless Slack output.
- 4 Run PF, initialize dynamics and calculate eigenvalues.
- 5 Store accepted samples; summarize rejection reasons.

Source: Authors' workflow and calculations using ANDES [2].

The generator separates data from rejection diagnostics

One function runs all sampled candidates

```
dataset, failure_counts = ss.generate_dataset(  
    samples_per_commitment=100,  
    random_seed=7,  
    show_progress=True,  
)
```

700 attempted

488 accepted

Returned objects

dataset

One row per feasible and successfully initialized operating point, including features, label and critical real part.

failure_counts

Counts rejected candidates by normalized reason, such as bus voltage, Slack limit or dynamic initialization failure.

Source: Authors' implementation in `small_signal_functions.py`.

Eight features match UC decisions and the operating profile

$$z = [u_2, u_3, u_6, P_{g,2}, P_{g,3}, P_{g,6}, P_{s,8}, P_L^{\text{total}}]$$

Commitment

$$u_2, u_3, u_6 \in \{0, 1\}$$

Which switchable synchronous generators are online.

Dispatch

$$P_{g,2}, P_{g,3}, P_{g,6}, P_{s,8}$$

SG active powers and the GFL solar injection.

Operating profile

$$P_L^{\text{total}}$$

Total active load; bus shares and power factors remain fixed.

Every feature is either a UC decision or a fixed profile value available to the UC.

Source: Authors' feature definition used across data, learning and UC.

Sampling combines coverage with boundary refinement

Sampling design

- 25% broad: $P_s \sim U(0, 100)$ MW and full online-SG ranges.
- 75% boundary: $P_s \sim U(92, 96)$ MW and P_g near $P_g^{\min}$.
- $P_L^{\text{total}} \in [178.96, 313.18]$ MW.
- Pre-screen: $50 \leq P_{\text{slack}}^{\text{approx}} \leq 200$ MW.

Fixed spatial distribution and power factor

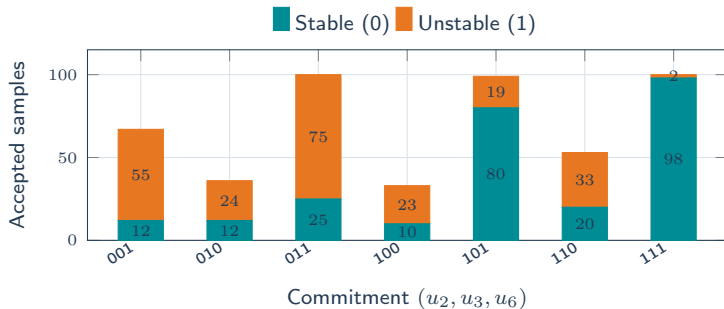
$$P_{l,i} = P_L^{\text{total}} \frac{P_{l,i}^{\text{base}}}{\sum_j P_{l,j}^{\text{base}}}$$

$$Q_{l,i} = P_{l,i} \frac{Q_{l,i}^{\text{base}}}{P_{l,i}^{\text{base}}}$$

Total load is sufficient as training feature because bus shares are fixed.

Source: Authors' sampling implementation in `sample_operating_points()`.

The dataset is balanced overall; commitment remains informative



Overall balance

257

stable (52.7%)

231

unstable (47.3%)

(0, 0, 1): 12 stable / 55
unstable

(1, 1, 1): 98 stable / 2
unstable

**Commitment is
informative, not sufficient.**

Source: Authors' calculations from outputs/small_signal_dataset.csv, $n = 488$.

Regression preserves distance from the stability boundary

The network predicts the continuous critical real part:

$$\hat{\alpha} = \mathcal{N}_{\theta}(z)$$

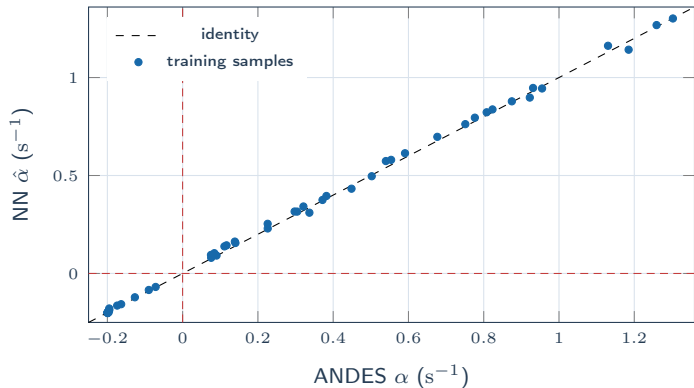


Training choices

- Commitments remain in $\{0, 1\}$.
- Every MW input is divided by 100 MVA.
- Linear(1) output with no activation; full-batch MSE.
- Adam: learning rate 3×10^{-3} , weight decay 10^{-4} .
- 1500 epochs on all 488 samples.

Source: Authors' implementation using PyTorch [7].

The trained regressor closely reproduces ANDES targets in sample



Source: Authors' calculations from the saved dataset and trained model.

Metric	Value
MAE	0.011470
RMSE	0.016628
R^2	0.998363
False-stable	0

Training-set metrics do not measure out-of-sample generalization.

Baseline UC exposes the cost–stability conflict

$$\begin{aligned} \min \quad & c_0 P_0 + c_g^\top P_g + c_u^\top u_g + c_{\text{curt}}(P_s^{\text{avail}} - P_s) \\ \text{s.t.} \quad & P_g^{\min} u_g \leq P_g \leq P_g^{\max} u_g, \\ & 50 \leq P_0 \leq 200, \\ & 0 \leq P_s \leq P_s^{\text{avail}}, \\ & P_0 + \mathbf{1}^\top P_g + P_s = P_L, \\ & \mathbf{1}^\top u_g \geq 1, \quad u_g \in \{0, 1\}^3. \end{aligned}$$

One-period tutorial scope

- no ramping;
- no startup/shutdown;
- no transmission limits; and
- no power losses in UC.

Bus-6 SG is cheapest, so economics favours the stability-challenging $(0, 0, 1)$ commitment.

Source: Authors' formulation using CVXPY [6].

The economic optimum is small-signal unstable

Profile: $P_L = 223.7$ MW, $P_s^{\text{avail}} = 95$ MW

Baseline UC result	Value
Objective	3901.00
Commitment	(0, 0, 1)
SG dispatch	(0, 0, 30) MW
Solar injection	95.00 MW
Planned Slack	98.70 MW
NN-predicted $\hat{\alpha}$	$+0.834004 \text{ s}^{-1}$
ANDES α	$+0.819129 \text{ s}^{-1}$
ANDES status	Unstable

$$\alpha > 0$$

UNSTABLE

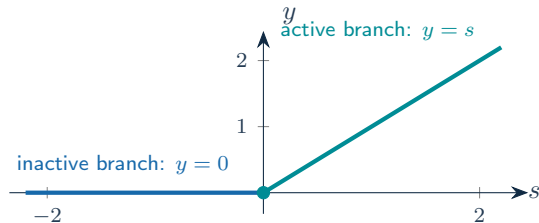
Conventional UC feasibility
does not imply dynamic
stability.

Source: Authors' solution using CVXPY [6] and verification using ANDES [2].

One bounded ReLU has an exact MILP representation

ReLU activation

$$s = Wz + b, \quad L \leq s \leq U, \quad y = \max(0, s)$$



Exact MILP constraints

$$\begin{aligned} y &\geq 0, & y &\geq s, \\ y &\leq U\delta, & y &\leq s - L(1 - \delta), \\ \delta &\in \{0, 1\}. \end{aligned}$$

Source: Adapted from Xu, Chu and Teng [1].

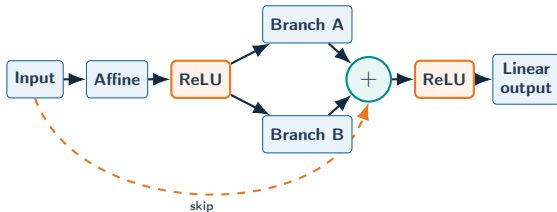
Network graphs make manual MILP conversion difficult

Sequential graph



Layers form one chain, so variables and constraints can be created in order.

Residual and branched graph

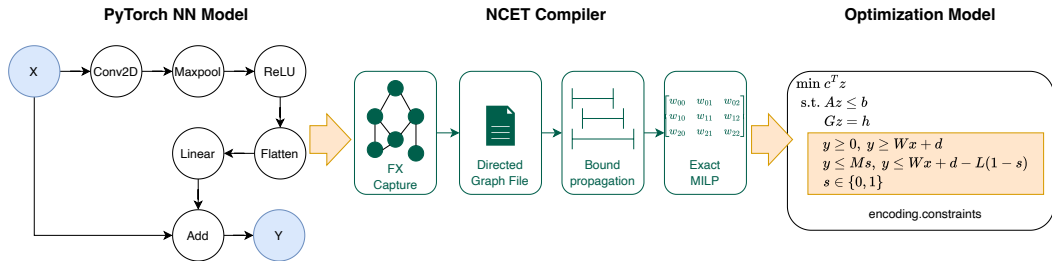


- Branches, tensor reuse, skip connections and merges break the simple layer-by-layer workflow.
- Every tensor needs consistent variables, bounds and graph connectivity.

NCET captures the graph and automatically generates consistent MILP constraints.

Sources: Xu, Chu and Teng [1]; NCET documentation [3].

NCET converts PyTorch graphs into exact MILP constraints



Exact, not approximate



Graph-aware, not sequential-only



Binary variables
reduction by default

Exact

Supported operators become equivalent CVXPY MILP constraints.

```
pip install ncet
```

Graph-aware

`torch.fx` preserves branches and residual connections.

github.com/xuwkk/ncet

Efficient

Bound propagation removes binaries for sign-stable ReLUs.

xuwkk.github.io/ncet/

Sources: Xu, Chu and Teng [1]; NCET documentation [3].

NCET bounds and compiles the trained graph

1. Define normalized bounds

```
commitment: [0, 1]
SG power:   [0, 0.5] pu
solar:      [0, 0.95] pu
total load: 2.237 pu, fixed
```

Bounds define the operating domain used for interval bound propagation and ReLU-state reduction.

Source: Authors' implementation using NCET [3].

2. Compile the PyTorch graph

```
nn_encoding = form_milp(
    regressor,
    Bounds(lower=nn_lower_bound,
            upper=nn_upper_bound),
    relu_binary_mode="reduced",
)
```

`form_milp()` traces the supported PyTorch graph, creates CVXPY variables and emits the equivalent mixed-integer constraints in one function call.

The encoded output then becomes a UC stability constraint

3. Link NCET input to UC variables

```
nn_inputs = list(nn_encoding.inputs.values())
nn_input = nn_inputs[0]

constraints += [
    nn_input == uc_feature_expression
]
```

The equality links the symbolic network input to commitment, dispatch, solar and total-load expressions in the UC.

4. Impose a stability margin

```
nn_outputs = list(
    nn_encoding.outputs.values()
)
predicted_critical_real = nn_outputs[0][0]

constraints += [
    predicted_critical_real <= -0.01
]
```

The negative margin asks the optimizer to stay on the predicted stable side of the boundary.

Exact network encoding does not remove regression error relative to ANDES.

Source: Authors' implementation using NCET [3].

The stability constraint changes commitment and cost

Result	Baseline UC	Stability-constrained UC
Objective	3901.00	4289.77
Commitment	(0, 0, 1)	(1, 0, 1)
SG dispatch	(0, 0, 30) MW	(10, 0, 30) MW
Solar injection	95.000 MW	94.702 MW
NN-predicted $\hat{\alpha}$	+0.834004	-0.010000
ANDES α	+0.819129	-0.003896
ANDES status	Unstable	Stable

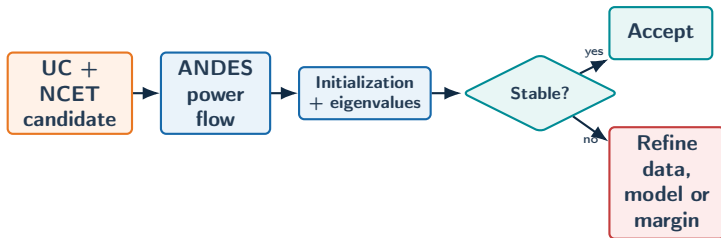
Commit bus-2 SG
at its 10-MW minimum

Curtail 0.298 MW
of additional solar

Pay 388.77 more
approximately 10.0%

Source: Authors' results using CVXPY [6], NCET [3] and ANDES [2].

ANDES independently validates the NN-selected candidate



Why verification matters

- UC uses lossless balance; ANDES includes AC losses.
- Baseline Slack: 98.70 planned vs 103.18 MW in ANDES.
- Constrained Slack: 89.00 planned vs 92.77 MW in ANDES.
- $\hat{\alpha} = -0.01$ in UC; ANDES gives $\alpha = -0.003896$.

The NN selects quickly; physics validates independently.

Source: Authors' optimized operating points and verification using ANDES [2].

Next extensions toward production use

Improve learning and sampling

- **Generalization:** validation and test sets before margin calibration.
- **Sampling:** adaptively add data near the learned boundary.
- **Controller dependence:** add controller features when they become decisions or scenarios.

Improve the power-system model

- **Network:** spatial load patterns and transmission limits.
- **Reactive feasibility:** SG and converter reactive-power limits.
- **Operations:** multiple periods, ramping, startup/shutdown and minimum times.

This demonstrates the methodology, not a production security guarantee.

Source: Authors' tutorial scope and limitations.

Simulation, learning and optimization close the loop

- 1 **ANDES** converts UC-style operating points into eigenvalue targets.
- 2 **Regression** retains distance from the stability boundary.
- 3 **NCET** converts the ReLU model into exact CVXPY constraints.
- 4 **Stability-constrained UC** trades operating cost for stable commitment.
- 5 **ANDES verification** remains essential because the surrogate is approximate.

Run the tutorial

[Neural-SmallSignal-UC-Tutorial](#)

[NCET on GitHub](#)

[ANDES documentation](#)

Next: validation, adaptive boundary sampling and network constraints.

References: methodology and core tools

- ① W. Xu, Z. Chu, and F. Teng, “Learning-Augmented Power System Operations: A Unified Optimization View,” *IEEE Transactions on Power Systems*, pp. 1–21, 2026. doi: 10.1109/TPWRS.2026.3726363.
- ② H. Cui, F. Li, and K. Tomsovic, “Hybrid Symbolic-Numeric Framework for Power System Modeling and Analysis,” *IEEE Transactions on Power Systems*, vol. 36, no. 2, pp. 1373–1384, Mar. 2021. doi: 10.1109/TPWRS.2020.3017019.
- ③ W. Xu, “NCET: Neural-network Constraint Embedding Toolkit,” 2026. github.com/xuwkk/ncet; [documentation](#).

References: models and software

- ④ F. Fang, M. Rahmatian, and K. Chilukuri, “Active and Reactive Power Priority Imposed by Current Limit Logic in Inverter-Based Generators,” *Proc. 2020 CIGRE Canada Conference*, Toronto, Oct. 2020, paper CIGRE-295.
- ⑤ Electric Power Research Institute, *Model User Guide for Generic Renewable Energy System Models*, Technical Update, EPRI Report 3002006525, June 2015.
- ⑥ S. Diamond and S. Boyd, “CVXPY: A Python-Embedded Modeling Language for Convex Optimization,” *Journal of Machine Learning Research*, vol. 17, no. 83, pp. 1–5, 2016.
- ⑦ A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” *Advances in Neural Information Processing Systems* 32, 2019.